

MACHINE LEARNING

DEEP LEARNING: FITTING NEURAL NETWORKS AND BACK-PROPAGATION

Sebastian Engelke

MASTER IN BUSINESS ANALYTICS



**UNIVERSITÉ
DE GENÈVE**

Loss functions for neural networks

- ▶ Neural networks are non-linear **parametric models**, with parameter vector $\theta = (\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(M+1)}, b^{(1)}, \dots, b^{(M+1)})$ consisting of the **weight matrices** and the **bias vectors**.

Loss functions for neural networks

- ▶ Neural networks are non-linear **parametric models**, with parameter vector $\theta = (\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(M+1)}, b^{(1)}, \dots, b^{(M+1)})$ consisting of the **weight matrices** and the **bias vectors**.
- ▶ For regression, we let $\hat{f} = \hat{f}_\theta$ denote the predictive model from the neural network based on the parameter set θ . Similarly, for classification with $\hat{f}_1, \dots, \hat{f}_q$.

Loss functions for neural networks

- ▶ Neural networks are non-linear **parametric models**, with parameter vector $\theta = (\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(M+1)}, b^{(1)}, \dots, b^{(M+1)})$ consisting of the **weight matrices** and the **bias vectors**.
- ▶ For regression, we let $\hat{f} = \hat{f}_\theta$ denote the predictive model from the neural network based on the parameter set θ . Similarly, for classification with $\hat{f}_1, \dots, \hat{f}_q$.
- ▶ Suppose now we have **training data** $(x_1, y_1), \dots, (x_n, y_n)$.
- ▶ For regression ($y_i \in \mathbb{R}$) we use the **squared-error loss**

$$J(\theta) = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2.$$

Loss functions for neural networks

- ▶ Neural networks are non-linear **parametric models**, with parameter vector $\theta = (\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(M+1)}, b^{(1)}, \dots, b^{(M+1)})$ consisting of the **weight matrices** and the **bias vectors**.
- ▶ For regression, we let $\hat{f} = \hat{f}_\theta$ denote the predictive model from the neural network based on the parameter set θ . Similarly, for classification with $\hat{f}_1, \dots, \hat{f}_q$.
- ▶ Suppose now we have **training data** $(x_1, y_1), \dots, (x_n, y_n)$.
- ▶ For regression ($y_i \in \mathbb{R}$) we use the **squared-error loss**

$$J(\theta) = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2.$$

- ▶ For classification ($y_i \in \{1, \dots, q\}$) we use **cross-entropy (deviance) loss**

$$J(\theta) = - \sum_{i=1}^n \sum_{j=1}^q \mathbf{1}_{\{y_i = j\}} \log \hat{f}_j(x_i),$$

which is the **negative log-likelihood** of the **multinomial distribution**.

Fitting of neural networks

- ▶ **Fitting neural networks** is an art (and an area of research) for itself.
- ▶ The problem of **minimizing** $J(\theta)$ does not have a closed form solution and is **non-convex**.

Fitting of neural networks

- ▶ **Fitting neural networks** is an art (and an area of research) for itself.
- ▶ The problem of **minimizing** $J(\theta)$ does not have a closed form solution and is **non-convex**.
- ▶ Since we have so many parameters, we are typically not interested in the **global/local minimum** of $J(\theta)$ since this would be an **overfit**.

Fitting of neural networks

- ▶ **Fitting neural networks** is an art (and an area of research) for itself.
- ▶ The problem of **minimizing** $J(\theta)$ does not have a closed form solution and is **non-convex**.
- ▶ Since we have so many parameters, we are typically not interested in the **global/local minimum** of $J(\theta)$ since this would be an **overfit**.
- ▶ Instead we will prevent overfitting by **regularization** or **early stopping** (later more).

Fitting of neural networks

- ▶ **Fitting neural networks** is an art (and an area of research) for itself.
- ▶ The problem of **minimizing** $J(\theta)$ does not have a closed form solution and is **non-convex**.
- ▶ Since we have so many parameters, we are typically not interested in the **global/local minimum** of $J(\theta)$ since this would be an **overfit**.
- ▶ Instead we will prevent overfitting by **regularization** or **early stopping** (later more).
- ▶ For now, we first consider how we can improve the initial model using **gradient descent**.

Fitting of neural networks

- ▶ **Fitting neural networks** is an art (and an area of research) for itself.
- ▶ The problem of **minimizing** $J(\theta)$ does not have a closed form solution and is **non-convex**.
- ▶ Since we have so many parameters, we are typically not interested in the **global/local minimum** of $J(\theta)$ since this would be an **overfit**.
- ▶ Instead we will prevent overfitting by **regularization** or **early stopping** (later more).
- ▶ For now, we first consider how we can improve the initial model using **gradient descent**.

Gradient descent for neural networks

- Start with an arbitrary initial value $\theta^{(0)}$.
- For $i = 1, 2, \dots$:
 - Compute the **gradient/derivative** of J at the position $\theta^{(i-1)}$:

$$\nabla_{\theta} J(\theta^{(i-1)}) = \left. \frac{\partial J(\theta)}{\partial \theta} \right|_{\theta=\theta^{(i-1)}}$$

- Update your estimate with **learning rate** $\alpha > 0$ to

$$\theta^{(i)} = \theta^{(i-1)} - \alpha \nabla_{\theta} J(\theta^{(i-1)}).$$

- Stop if a **stopping criterion** is satisfied.

Fitting of neural networks

- ▶ **Fitting neural networks** is an art (and an area of research) for itself.
- ▶ The problem of **minimizing** $J(\theta)$ does not have a closed form solution and is **non-convex**.
- ▶ Since we have so many parameters, we are typically not interested in the **global/local minimum** of $J(\theta)$ since this would be an **overfit**.
- ▶ Instead we will prevent overfitting by **regularization** or **early stopping** (later more).
- ▶ For now, we first consider how we can improve the initial model using **gradient descent**.

Gradient descent for neural networks

- Start with an arbitrary initial value $\theta^{(0)}$.
- For $i = 1, 2, \dots$:
 - Compute the **gradient/derivative** of J at the position $\theta^{(i-1)}$:

$$\nabla_{\theta} J(\theta^{(i-1)}) = \left. \frac{\partial J(\theta)}{\partial \theta} \right|_{\theta=\theta^{(i-1)}}$$

- Update your estimate with **learning rate** $\alpha > 0$ to

$$\theta^{(i)} = \theta^{(i-1)} - \alpha \nabla_{\theta} J(\theta^{(i-1)}).$$

- Stop if a **stopping criterion** is satisfied.

Thus, all we need are the **gradients** $\nabla_{\theta} J(\theta)$!

Computing the gradient: back-propagation

- ▶ In the sequel we consider only **regression**; classification is similar.
- ▶ The gradient $\nabla J(\theta)$ at some parameter vector θ is the sum of the gradients over all training samples:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (\hat{f}_{\theta}(x_i) - y_i)^2.$$

Computing the gradient: back-propagation

- ▶ In the sequel we consider only **regression**; classification is similar.
- ▶ The gradient $\nabla J(\theta)$ at some parameter vector θ is the sum of the gradients over all training samples:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (\hat{f}_{\theta}(x_i) - y_i)^2.$$

- ▶ We can thus consider the summands separately, and for now we consider only the **loss contribution** of one training sample (x, y)

$$L(\theta) = (\hat{f}_{\theta}(x) - y)^2.$$

- ▶ **Back-propagation** is a way of efficiently deriving the gradient using the **chain-rule** for differentiation.

Computing the gradient: back-propagation

- ▶ In the sequel we consider only **regression**; classification is similar.
- ▶ The gradient $\nabla J(\theta)$ at some parameter vector θ is the sum of the gradients over all training samples:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (\hat{f}_{\theta}(x_i) - y_i)^2.$$

- ▶ We can thus consider the summands separately, and for now we consider only the **loss contribution** of one training sample (x, y)

$$L(\theta) = (\hat{f}_{\theta}(x) - y)^2.$$

- ▶ **Back-propagation** is a way of efficiently deriving the gradient using the **chain-rule** for differentiation.
- ▶ Recall: for two functions f and g , let $h(x) = f(g(x))$, the chain-rule is

$$\left. \frac{\partial h(x)}{\partial x} \right|_{x=x_0} = \left. \frac{\partial f(z)}{\partial z} \right|_{z=g(x_0)} \cdot \left. \frac{\partial g(x)}{\partial x} \right|_{x=x_0},$$

or in short-hand notation with $z = g(x)$

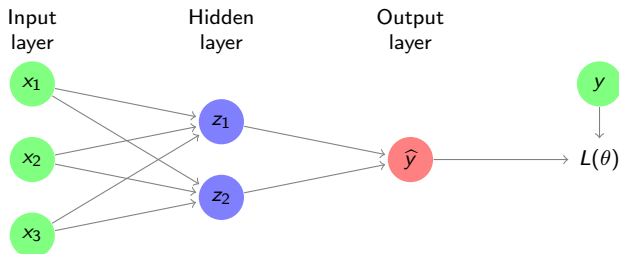
$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial x}$$

Back-propagation: the forward pass

- ▶ Fix the training sample $x = (x_1, \dots, x_p)$ and y , and also the parameter vector θ consisting of weights and biases where we want to compute the gradient of $L(\theta)$.
- ▶ In order to know all the values of all the hidden neurons, we first have to do a **forward pass**.
- ▶ For this, we start with the **activations** $(x_1, \dots, x_p)$ of the inputs $(X_1, \dots, X_p)$ and then successively compute the activations z_i of all neurons of the hidden layers and the predicted value

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_{\theta}(x) = b^{(2)} + w^{(2)\top} z.$$

- ▶ This allows us to compute **the error** for this θ on that training sample $L(\theta) = (\hat{y} - y)^2$.

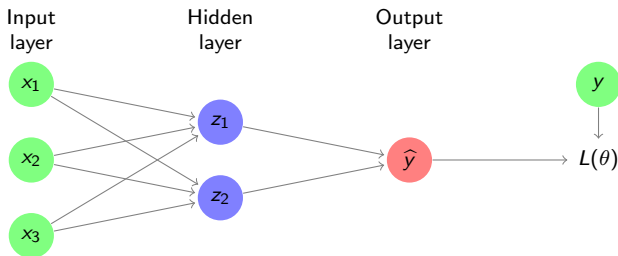


Back-propagation

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_\theta(x) = b^{(2)} + w^{(2)\top}z, \quad L(\theta) = (\hat{y} - y)^2.$$

- ▶ To obtain the gradient, we compute the **derivatives** of L with respect to **all parameters**.
- ▶ Let us start with the weight vector $w^{(2)} = (w_1^{(2)}, w_2^{(2)})$:

$$\frac{\partial L}{\partial w_1^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1^{(2)}}$$

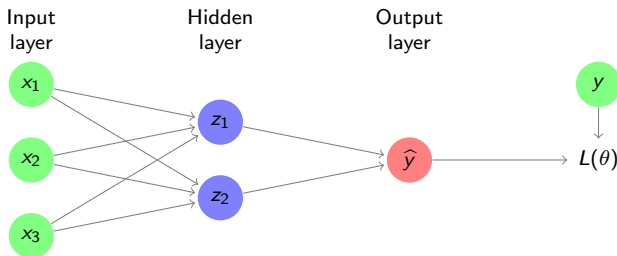


Back-propagation

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_{\theta}(x) = b^{(2)} + w^{(2)\top}z, \quad L(\theta) = (\hat{y} - y)^2.$$

- ▶ To obtain the gradient, we compute the **derivatives** of L with respect to **all parameters**.
- ▶ Let us start with the weight vector $w^{(2)} = (w_1^{(2)}, w_2^{(2)})$:

$$\frac{\partial L}{\partial w_1^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1^{(2)}} = 2(\hat{y} - y)z_1$$



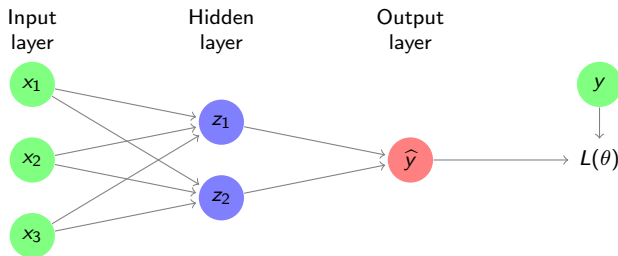
Back-propagation

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_\theta(x) = b^{(2)} + w^{(2)\top}z, \quad L(\theta) = (\hat{y} - y)^2.$$

- ▶ To obtain the gradient, we compute the **derivatives** of L with respect to **all parameters**.
- ▶ Let us start with the weight vector $w^{(2)} = (w_1^{(2)}, w_2^{(2)})$:

$$\frac{\partial L}{\partial w_1^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_1^{(2)}} = 2(\hat{y} - y)z_1$$

$$\frac{\partial L}{\partial w_2^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2^{(2)}} = 2(\hat{y} - y)z_2$$

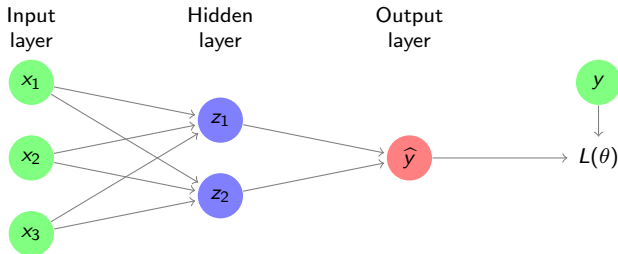


Back-propagation

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_{\theta}(x) = b^{(2)} + w^{(2)\top}z, \quad L(\theta) = (\hat{y} - y)^2.$$

► For the weight $w_{11}^{(1)}$:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}^{(1)}}$$

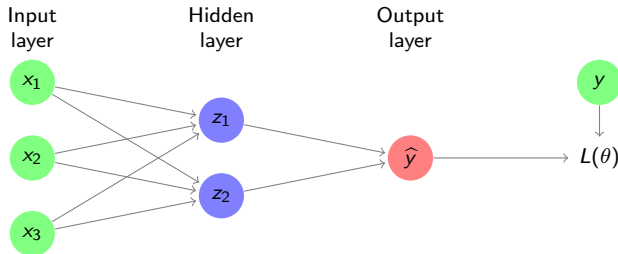


Back-propagation

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_{\theta}(x) = b^{(2)} + w^{(2)\top}z, \quad L(\theta) = (\hat{y} - y)^2.$$

► For the weight $w_{11}^{(1)}$:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}^{(1)}} = 2(\hat{y} - y) \cdot w_1^{(2)} \cdot g'(b_1^{(1)} + w_1^{(1)\top}x)x_1$$



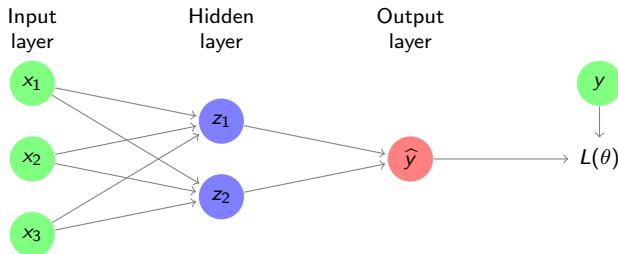
Back-propagation

$$z = g(b^{(1)} + \mathbf{W}^{(1)}x), \quad \hat{y} = \hat{f}_\theta(x) = b^{(2)} + w^{(2)\top}z, \quad L(\theta) = (\hat{y} - y)^2.$$

► For the weight $w_{11}^{(1)}$:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}^{(1)}} = 2(\hat{y} - y) \cdot w_1^{(2)} \cdot g'(b_1^{(1)} + w_1^{(1)\top}x)x_1$$

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_i} \cdot \frac{\partial z_i}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top}x)x_j$$



Back-propagation

- Now write

$$\frac{\partial L}{\partial w_i^{(2)}} = 2(\hat{y} - y)z_i = \delta_i^{(2)}z_i,$$

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x)x_j = \delta_i^{(1)}x_j$$

- The quantities $\delta_i^{(2)}$ and $\delta_i^{(1)}$ are called the ‘‘errors’’ from the current model corresponding to θ at the output and hidden layer neurons, respectively.

Back-propagation

- ▶ Now write

$$\frac{\partial L}{\partial w_i^{(2)}} = 2(\hat{y} - y)z_i = \delta^{(2)}z_i,$$

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x)x_j = \delta_i^{(1)}x_j$$

- ▶ The quantities $\delta^{(2)}$ and $\delta_i^{(1)}$ are called the ‘‘errors’’ from the current model corresponding to θ at the output and hidden layer neurons, respectively.
- ▶ They satisfy

$$\delta_i^{(1)} = \delta^{(2)} \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x). \quad (1)$$

Note: for the **ReLU activation** $g(x) = \max\{0, x\}$ we have $g'(x) = \mathbf{1}\{x > 0\}$.

Back-propagation

- Now write

$$\frac{\partial L}{\partial w_i^{(2)}} = 2(\hat{y} - y)z_i = \delta_i^{(2)}z_i,$$

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x)x_j = \delta_i^{(1)}x_j$$

- The quantities $\delta_i^{(2)}$ and $\delta_i^{(1)}$ are called the ‘‘errors’’ from the current model corresponding to θ at the output and hidden layer neurons, respectively.
- They satisfy

$$\delta_i^{(1)} = \delta_i^{(2)} \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x). \quad (1)$$

Note: for the **ReLU activation** $g(x) = \max\{0, x\}$ we have $g'(x) = \mathbf{1}\{x > 0\}$.

- Back-propagation is implemented with a **two-pass algorithm**:
 - 1) Compute all z_i and $\hat{y}$ in the **forward pass**.
 - 2) In the **backward-pass**, the error $\delta_i^{(2)}$ are computed and then **back-propagated** via (1) to obtain the errors $\delta_i^{(1)}$, and then the partial derivatives required for the gradient.

Back-propagation

- Now write

$$\frac{\partial L}{\partial w_i^{(2)}} = 2(\hat{y} - y)z_i = \delta_i^{(2)}z_i,$$

$$\frac{\partial L}{\partial w_{ij}^{(1)}} = 2(\hat{y} - y) \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x)x_j = \delta_i^{(1)}x_j$$

- The quantities $\delta_i^{(2)}$ and $\delta_i^{(1)}$ are called the ‘‘errors’’, from the current model corresponding to θ at the output and hidden layer neurons, respectively.
- They satisfy

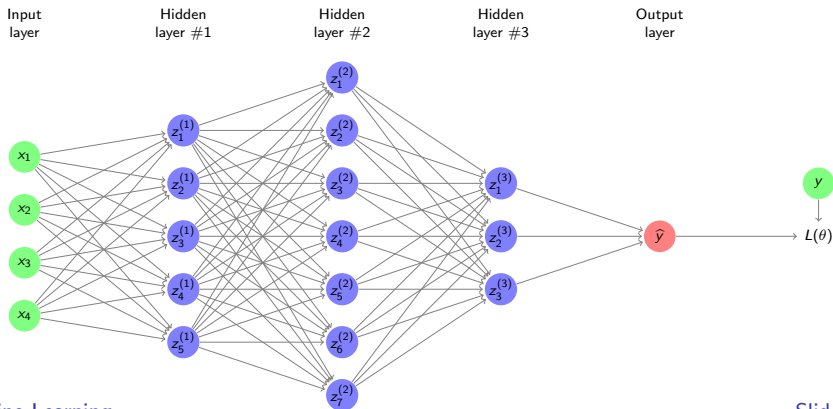
$$\delta_i^{(1)} = \delta_i^{(2)} \cdot w_i^{(2)} \cdot g'(b_i^{(1)} + w_i^{(1)\top} x). \quad (1)$$

Note: for the **ReLU activation** $g(x) = \max\{0, x\}$ we have $g'(x) = \mathbf{1}\{x > 0\}$.

- Back-propagation is implemented with a **two-pass algorithm**:
 - 1) Compute all z_i and $\hat{y}$ in the **forward pass**.
 - 2) In the **backward-pass**, the error $\delta_i^{(2)}$ are computed and then **back-propagated** via (1) to obtain the errors $\delta_i^{(1)}$, and then the partial derivatives required for the gradient.
- The equations for the derivatives with respect to the biases are similar.

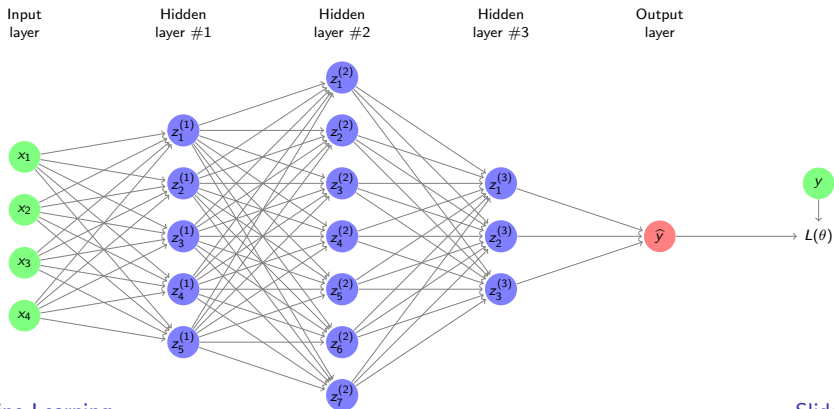
Advantages of back-propagation

- ▶ Back-propagation for **deep networks** works similarly, where we have error vectors $\delta^{(1)}, \dots, \delta^{(M+1)}$ at all M hidden layers and the output layer.
- ▶ Back-propagation is very **efficient** since the quantities $\delta^{(m)}$ can be stored and re-used, especially if the network is large.



Advantages of back-propagation

- ▶ Back-propagation for **deep networks** works similarly, where we have error vectors $\delta^{(1)}, \dots, \delta^{(M+1)}$ at all M hidden layers and the output layer.
- ▶ Back-propagation is very **efficient** since the quantities $\delta^{(m)}$ can be stored and re-used, especially if the network is large.
- ▶ Only the **local derivatives** at each node are required.
- ▶ It can be efficiently **parallelized** on GPUs.



Batch optimization

- ▶ Recall that the gradient required for **gradient descent** is the sum of the gradients over all training samples:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (y_i - \hat{f}_{\theta}(x_i))^2. \quad (1)$$

- ▶ For **each training sample** (x_i, y_i) we can compute the gradient via **back-propagation**.
- ▶ In order to improve our error function $J(\theta)$ we have two possibilities.

Batch optimization

- ▶ Recall that the gradient required for **gradient descent** is the sum of the gradients over all training samples:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (y_i - \hat{f}_{\theta}(x_i))^2. \quad (1)$$

- ▶ For **each training sample** (x_i, y_i) we can compute the gradient via **back-propagation**.
- ▶ In order to improve our error function $J(\theta)$ we have two possibilities.

1. Batch optimization and gradient descent

- ▶ Use the **batch** of all training samples $(x_1, y_1), \dots, (x_n, y_n)$ to compute the gradient in (1).
- ▶ This amounts to standard gradient descent and is called **batch-optimization**.

Stochastic gradient descent

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (y_i - \hat{f}_{\theta}(x_i))^2. \quad (1)$$

2. Minibatch optimization and stochastic gradient descent

- ▶ Modern problems often have **billions** of training samples $n \approx 10^9$, making a **single gradient step** prohibitively long.
- ▶ Randomly divide the training data in n/m **minibatches** of size m .
- ▶ The minibatch size m is usually **much smaller** than n , ranging from 1 to a few hundred.
- ▶ In each step, **stochastic gradient descent** uses only $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$ from a minibatch to approximate the gradient (1) for updating θ by

$$\nabla_{\theta} J(\theta) \approx \sum_{i=1}^m \nabla_{\theta} (y^{(i)} - \hat{f}_{\theta}(x^{(i)}))^2.$$

Stochastic gradient descent

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (y_i - \hat{f}_{\theta}(x_i))^2. \quad (1)$$

2. Minibatch optimization and stochastic gradient descent

- ▶ Modern problems often have **billions** of training samples $n \approx 10^9$, making a **single gradient step** prohibitively long.
- ▶ Randomly divide the training data in n/m **minibatches** of size m .
- ▶ The minibatch size m is usually **much smaller** than n , ranging from 1 to a few hundred.
- ▶ In each step, **stochastic gradient descent** uses only $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$ from a minibatch to approximate the gradient (1) for updating θ by

$$\nabla_{\theta} J(\theta) \approx \sum_{i=1}^m \nabla_{\theta} (y^{(i)} - \hat{f}_{\theta}(x^{(i)}))^2.$$

- ▶ Minibatch optimization takes **many small steps** that are possibly not optimal but **very fast** to compute.
- ▶ Batch optimization takes **few optimal steps** that are **much slower** and possibly infeasible for large data sets.

Stochastic gradient descent

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (y_i - \hat{f}_{\theta}(x_i))^2. \quad (1)$$

2. Minibatch optimization and stochastic gradient descent

- ▶ Modern problems often have **billions** of training samples $n \approx 10^9$, making a **single gradient step** prohibitively long.
- ▶ Randomly divide the training data in n/m **minibatches** of size m .
- ▶ The minibatch size m is usually **much smaller** than n , ranging from 1 to a few hundred.
- ▶ In each step, **stochastic gradient descent** uses only $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$ from a minibatch to approximate the gradient (1) for updating θ by

$$\nabla_{\theta} J(\theta) \approx \sum_{i=1}^m \nabla_{\theta} (y^{(i)} - \hat{f}_{\theta}(x^{(i)}))^2.$$

- ▶ Minibatch optimization takes **many small steps** that are possibly not optimal but **very fast** to compute.
- ▶ Batch optimization takes **few optimal steps** that are **much slower** and possibly infeasible for large data sets.
- ▶ Often, training samples are **similar** or **redundant** (e.g., similar images), then using minibatches discards less information.

Stochastic gradient descent

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^n \nabla_{\theta} (y_i - \hat{f}_{\theta}(x_i))^2. \quad (1)$$

2. Minibatch optimization and stochastic gradient descent

- ▶ Modern problems often have **billions** of training samples $n \approx 10^9$, making a **single gradient step** prohibitively long.
- ▶ Randomly divide the training data in n/m **minibatches** of size m .
- ▶ The minibatch size m is usually **much smaller** than n , ranging from 1 to a few hundred.
- ▶ In each step, **stochastic gradient descent** uses only $(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})$ from a minibatch to approximate the gradient (1) for updating θ by

$$\nabla_{\theta} J(\theta) \approx \sum_{i=1}^m \nabla_{\theta} (y^{(i)} - \hat{f}_{\theta}(x^{(i)}))^2.$$

- ▶ Minibatch optimization takes **many small steps** that are possibly not optimal but **very fast** to compute.
- ▶ Batch optimization takes **few optimal steps** that are **much slower** and possibly infeasible for large data sets.
- ▶ Often, training samples are **similar** or **redundant** (e.g., similar images), then using minibatches discards less information.
- ▶ A **training epoch** is one iteration over the **entire training set**.